

Stop Building Dumb Agents: Easy Database CRUD for Truly Intelligent Agents



Launch Intelligence
10.9K subscribers

Join

Subscribe

46



Share

Ask

Download



1,428 views Apr 29, 2025 [Members first on March 6, 2025](#) #AI #APIs #Python

Join AI Dev Skool & Launch Your AI Startup Today! <https://skool.com/ai-software-developers> is the community for founders, builders, and AI innovators ready to take their projects to the next level.

AI agents are powerful, but without structured data access, they lack reliability and scalability. In this tutorial, we'll integrate Supabase (PostgreSQL) with PydanticAI to create database-connected AI agents that can store, retrieve, and manage structured data efficiently.

What You'll Learn:

- ✓ Why databases are essential for AI agents
- ✓ How to use Supabase as a relational database for AI workflows
- ✓ Implementing CRUD (Create, Read, Update, Delete) operations
- ✓ Building context-aware database tools with PydanticAI
- ✓ Best practices for scalability, security, and performance

Why Database Agents Matter:

AI agents need persistent storage to handle:

- ✓ Data consistency & reliability for better decision-making
- ✓ Scalability to manage increasing data loads
- ✓ Security & compliance for enterprise and regulatory standards
- ✓ Efficient querying & retrieval using SQL
- ✓ Long-term storage & analytics for AI model improvement

Topics Covered:

- ✦ Setting up Supabase & connecting it to PydanticAI
- ✦ Creating database tools for querying, inserting, updating, and deleting data
- ✦ Handling error management and performance optimization
- ✦ Real-world demo: Building a CRUD-enabled AI agent

Full-Stack Agentic AI Masterclass Series:

- Part 1: Introduction
- Part 2: The AI framework landscape in 2025
- Part 3: Building effective agents - principles & practice
- Part 4: Prompt Engineering for effective agents
- Part 5: [Your AI Agents Are Useless Without This: P...](#)
- Part 6: Building UIs for your agents
- Part 7: [Stop Building Dumb Agents: Easy Database C...](#)
- Part 7.5: [AI That Writes SQL: Automate Queries with ...](#)
- Part 8: RAG
- Part 9: Integrating PydanticAI with N8N
- Part 10: Containers and deployment
- Part 11: E2E Final Project - customer support agent with PydanticAI



Full-Stack Agentic AI with PydanticAI

- The framework landscape in 2025
- Building effective agents - principles & practice
- Prompt Engineering for Better Results
- Building agent APIs - PydanticAI, FastAPI
- Building agent UI - PydanticAI, Streamlit
- Building DB agents with PydanticAI
- RAG with PydanticAI
- Integrating PydanticAI with N8N
- Containers and deployment
- Project: Customer support agent w/ PydanticAI



Database Agents

Full-Stack Agentic AI Masterclass

AI SOFTWARE DEV

Skool
Channel
Newsletter

WHY DB Agents?

Business enablers

1. **Data Consistency & Reliability** – Storing structured data in a database ensures that AI agents have a single source of truth, reducing errors and inconsistencies in decision-making.
2. **Scalability** – Databases allow AI agents to handle increasing volumes of structured data efficiently, supporting business growth without performance bottlenecks.
3. **Compliance & Security** – Many industries require data retention, auditability, and strict access controls. Databases provide encryption, authentication, and logging to meet regulatory standards.



WHY DB Agents?

Technical reasons

1. **Efficient Querying & Retrieval** – Unlike storing data in files or volatile memory, databases allow AI agents to perform complex queries, aggregations, and searches efficiently with SQL.
2. **Integration with External Systems** – AI agents often need to interact with CRM, ERP, and analytics tools. Databases provide structured, standardized data access, making integrations seamless.
3. **Long-Term Storage & Analysis** – AI agents improve when they have access to historical data for trend analysis, training, and fine-tuning—something databases handle far better than in-memory storage.



What We're Building

CRUD Agent with PydanticAI and Supabase

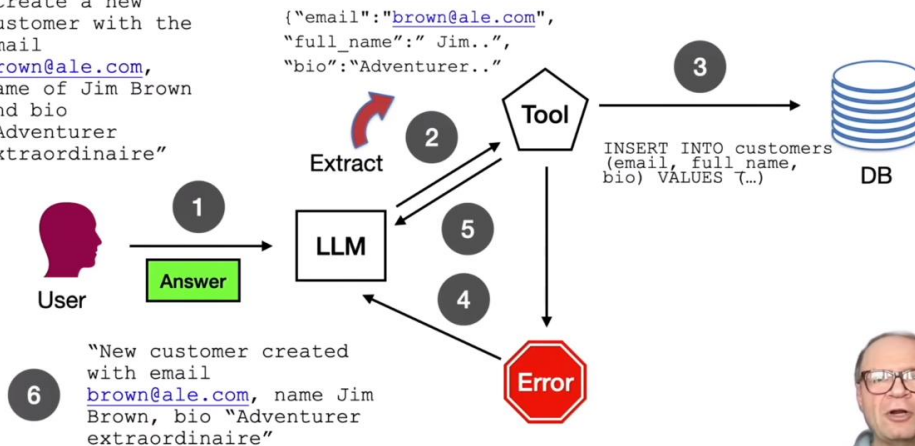
CRUD Operations

1. **Create a record** – a database insert tool to create a new customer
2. **Retrieve a record** – a database query tool to search for and return a customer
3. **Update a record** – a database tool to find a customer by email and update the record
4. **Delete a record** – a database tool to find a customer by email and delete the record



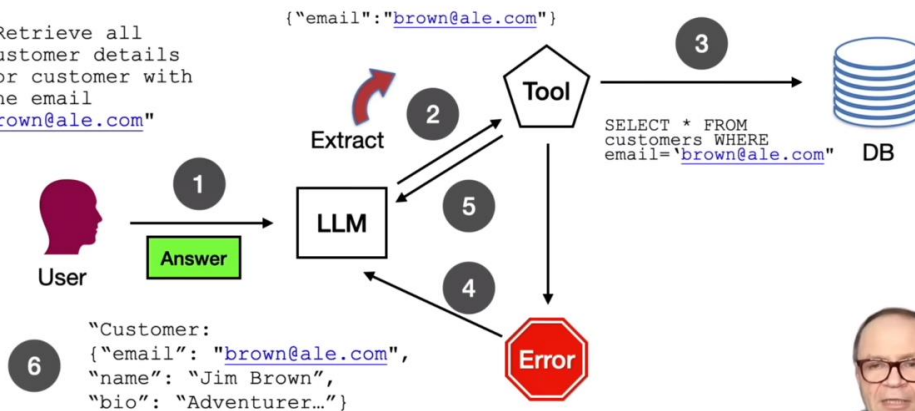
Agent Design - Create

"Create a new customer with the email brown@ale.com, name of Jim Brown and bio "Adventurer extraordinaire"



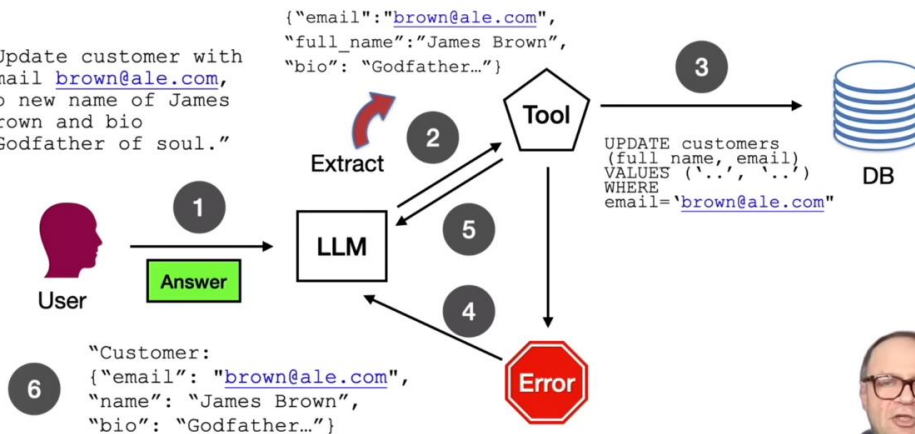
Agent Design - Retrieve

"Retrieve all customer details for customer with the email brown@ale.com"

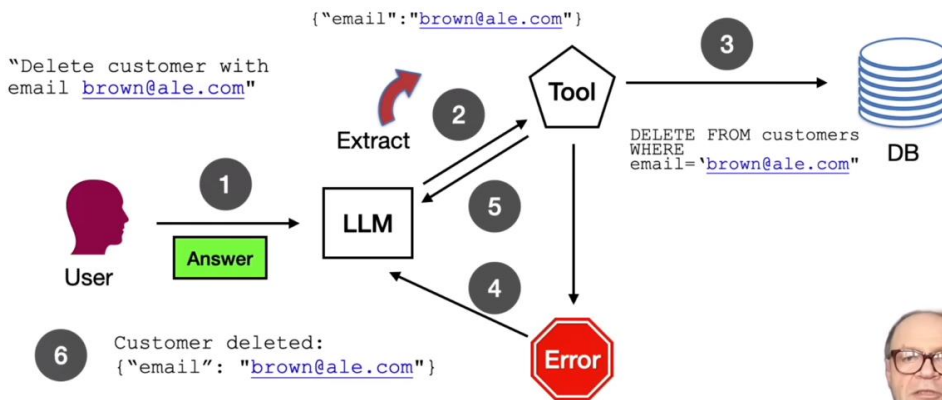


Agent Design - Update

"Update customer with email brown@ale.com, to new name of James Brown and bio "Godfather of soul."



Agent Design - Delete



Coding Tutorial

Database Agents with PydanticAI

1. Download and install **Docker Desktop**
2. Download and install **Supabase**
3. Create and configure the **customers** table – Create table, Create unique column index (email), Disable RLS
4. Create **basic agent** – write a basic agent with Supabase client access and DB seed function
5. Create **CRUD tools** – create, retrieve, update, delete tools for Supabase
6. **Test agent** – perform basic CRUD tests

Supabase setup

The screenshot shows the Supabase documentation page for Local Development & CLI. The page includes a sidebar with navigation links and a main content area with a quickstart guide.

Local Development & CLI

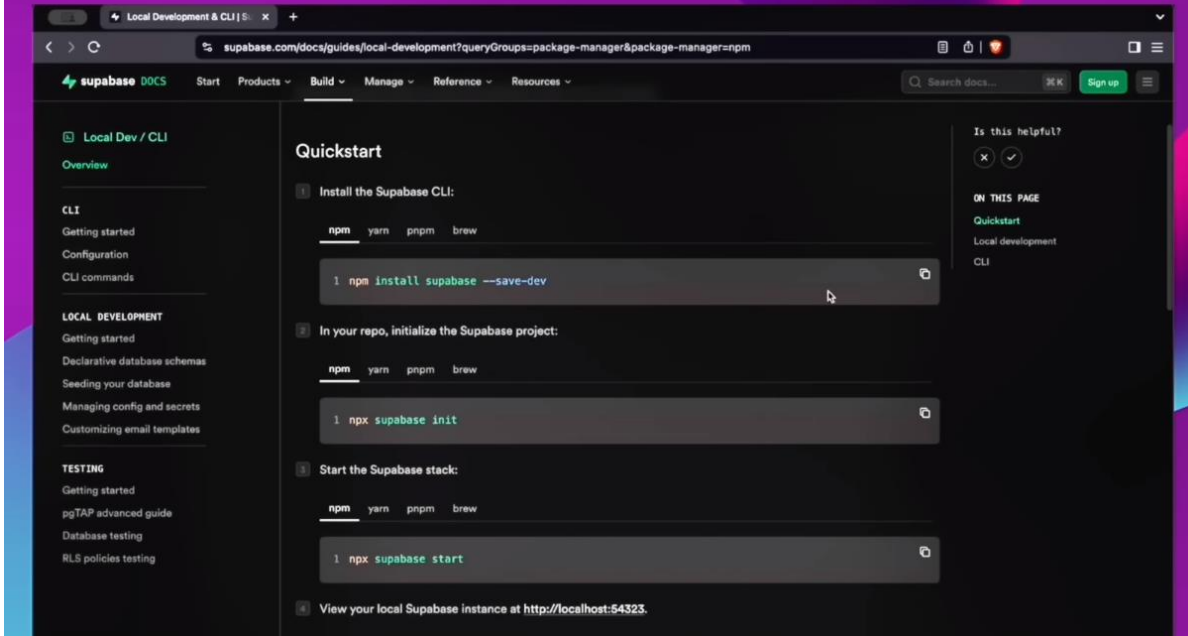
Learn how to develop locally and use the Supabase CLI

Develop locally while running the Supabase stack on your machine.

Quickstart

1. Install the Supabase CLI:
`npm install supabase --save-dev`
2. In your repo, initialize the Supabase project:
`npm run supabase init`

Supabase setup



Supabase setup

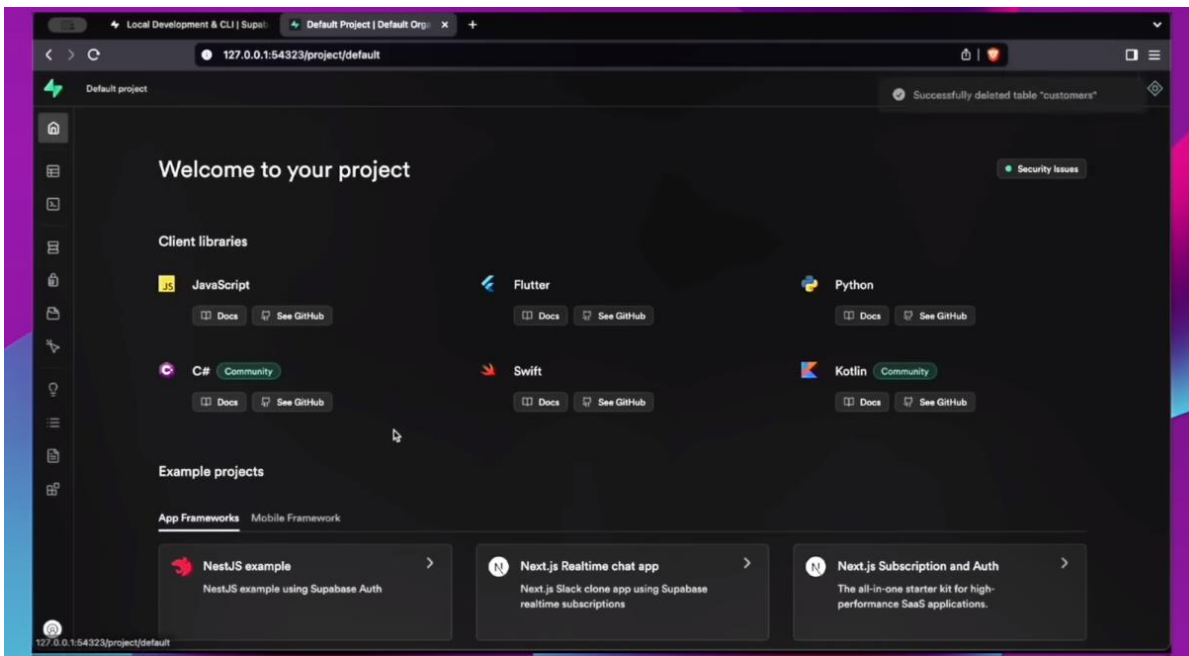
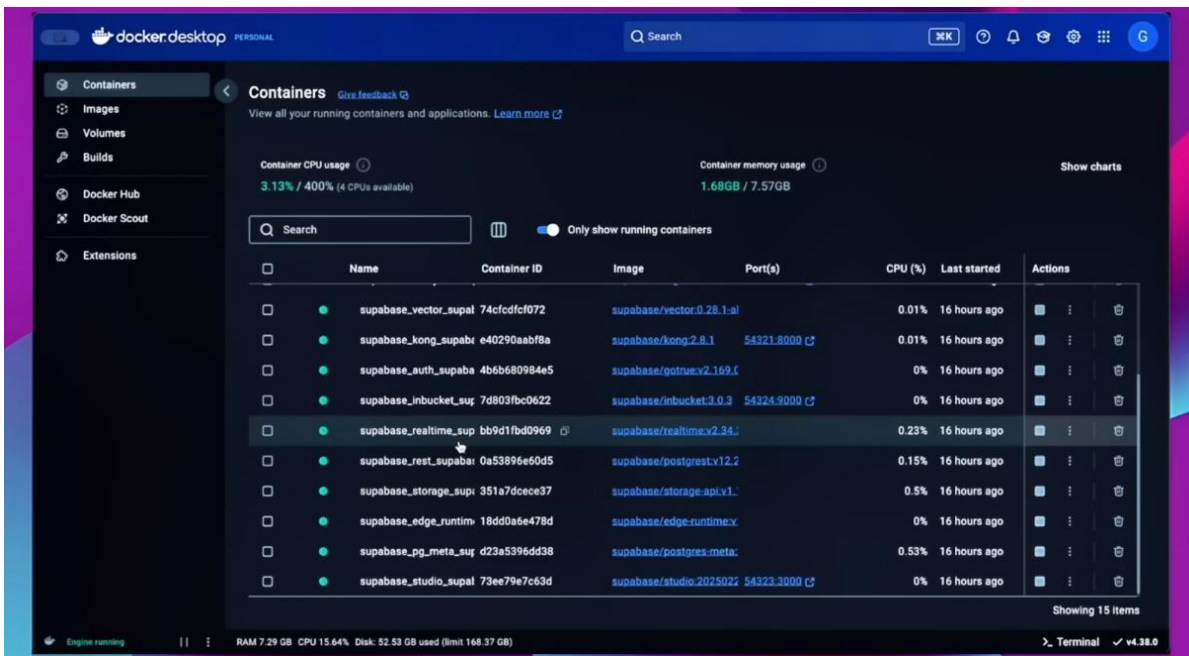
```
11:56:20 ~/dev/supabase $ npm install supabase --save-dev
up to date, audited 67 packages in 860ms
18 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
11:59:46 ~/dev/supabase $
```

```
11:59:46 ~/dev/supabase $ npx supabase init
failed to create config file: open supabase/config.toml: file exists
Run supabase init --force to overwrite existing config file.
12:00:10 ~/dev/supabase $ supabase init --force
Generate VS Code settings for Deno? [y/N]
Generate IntelliJ Settings for Deno? [y/N]
Finished supabase init.
A new version of Supabase CLI is available: v2.15.8 (currently installed v2.12.1)
We recommend updating regularly for new features and bug fixes: https://supabase.com/docs/guides/cli/getting-started#u
pdating-the-supabase-cli
12:00:30 ~/dev/supabase $
```

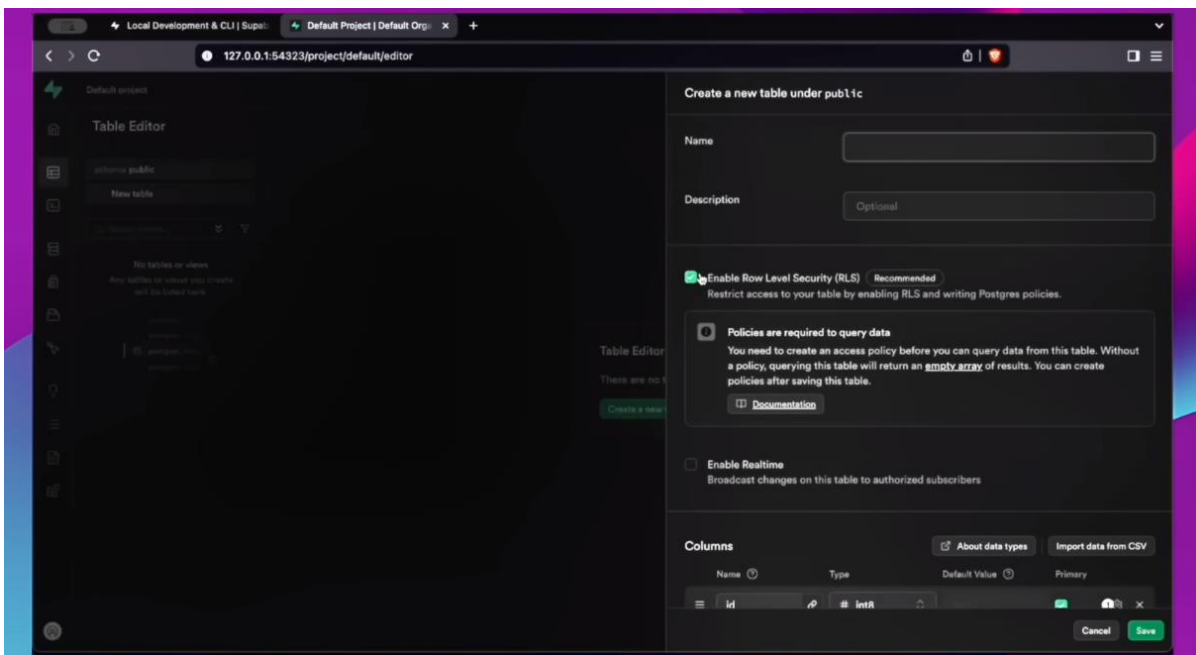
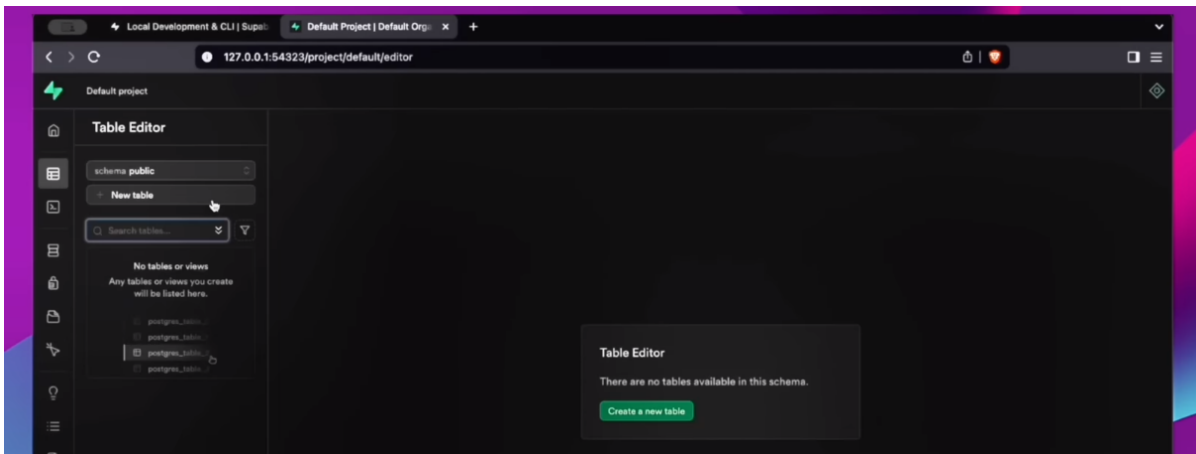
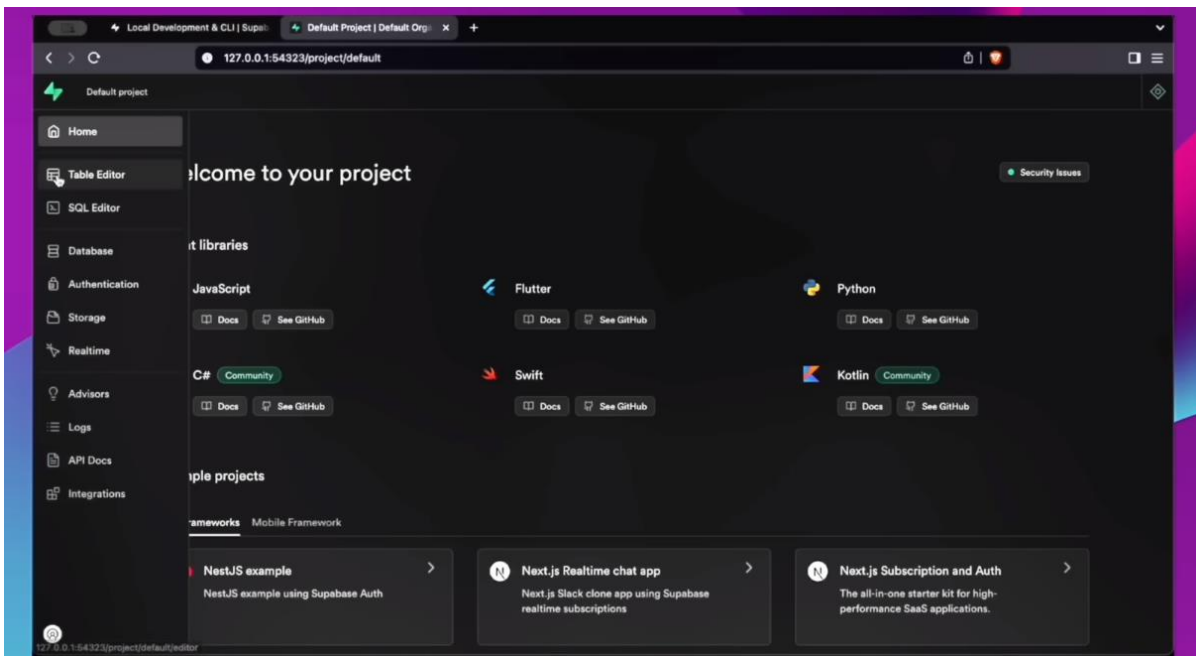
```
12:00:30 ~/dev/supabase $ npx supabase start
supabase start is already running.
Stopped services: [supabase_imgproxy_supabase supabase_pooler_supabase]
supabase local development setup is running.

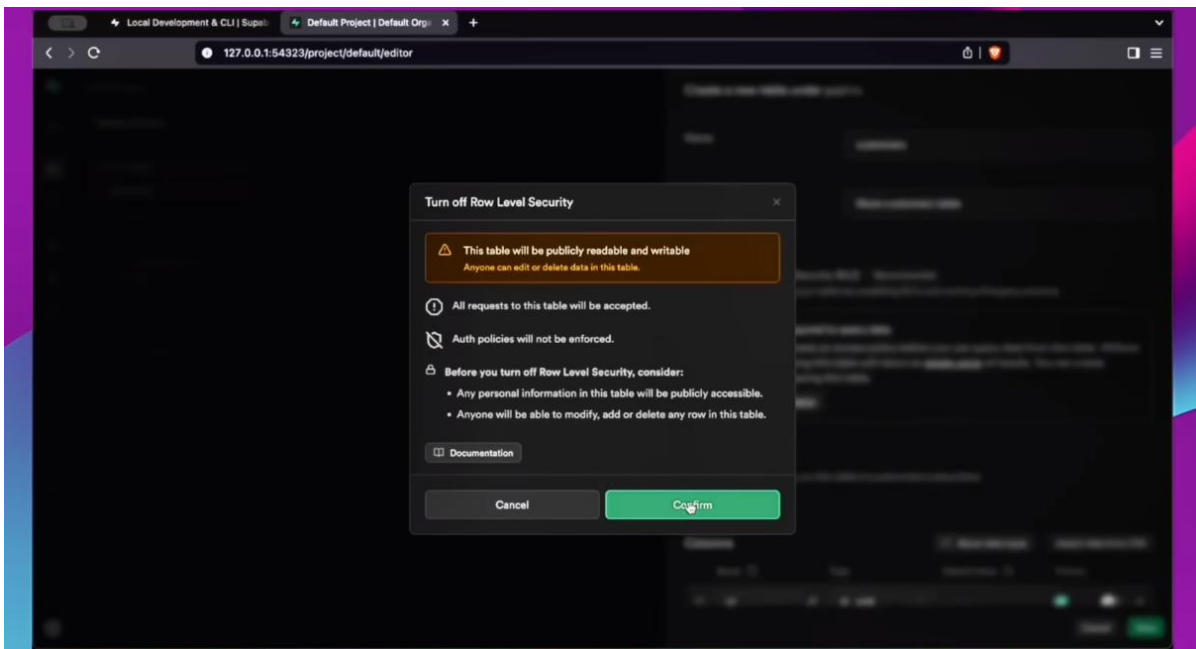
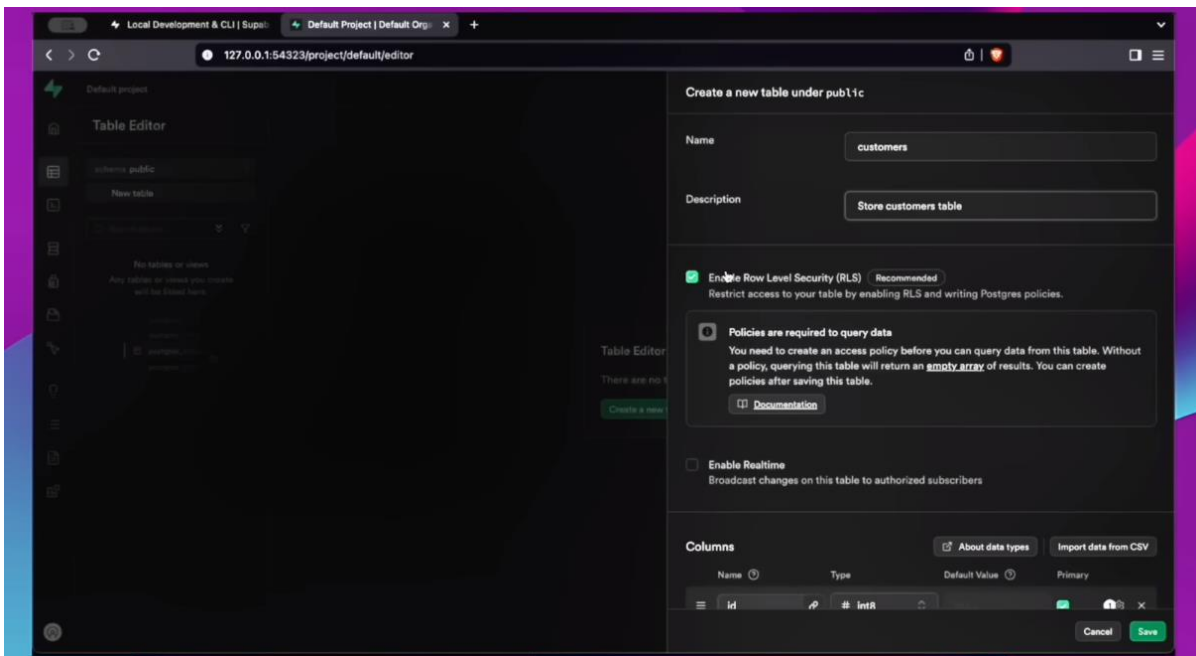
API URL: http://127.0.0.1:54321
GraphQL URL: http://127.0.0.1:54321/graphql/v1
S3 Storage URL: http://127.0.0.1:54321/storage/v1/s3
DB URL: postgresql://postgres:postgres@127.0.0.1:54322/postgres
Studio URL: http://127.0.0.1:54323
Inbucket URL: http://127.0.0.1:54324
JWT secret: super-secret-jwt-token-with-at-least-32-characters-long
anon key: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZS1kZW1vIiwicm9sZSI6ImFub24iLCJleHAiOjE5ODM0MTI5OTZ9.CRXP1A7W0e0JeXxjNni43kdQwgnWNRiellDMblyTn_I0
service_role key: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZS1kZW1vIiwicm9sZSI6ImFub24iLCJleHAiOjE5ODM0MTI5OTZ9.CRXP1A7W0e0JeXxjNni43kdQwgnWNRiellDMblyTn_I0
```

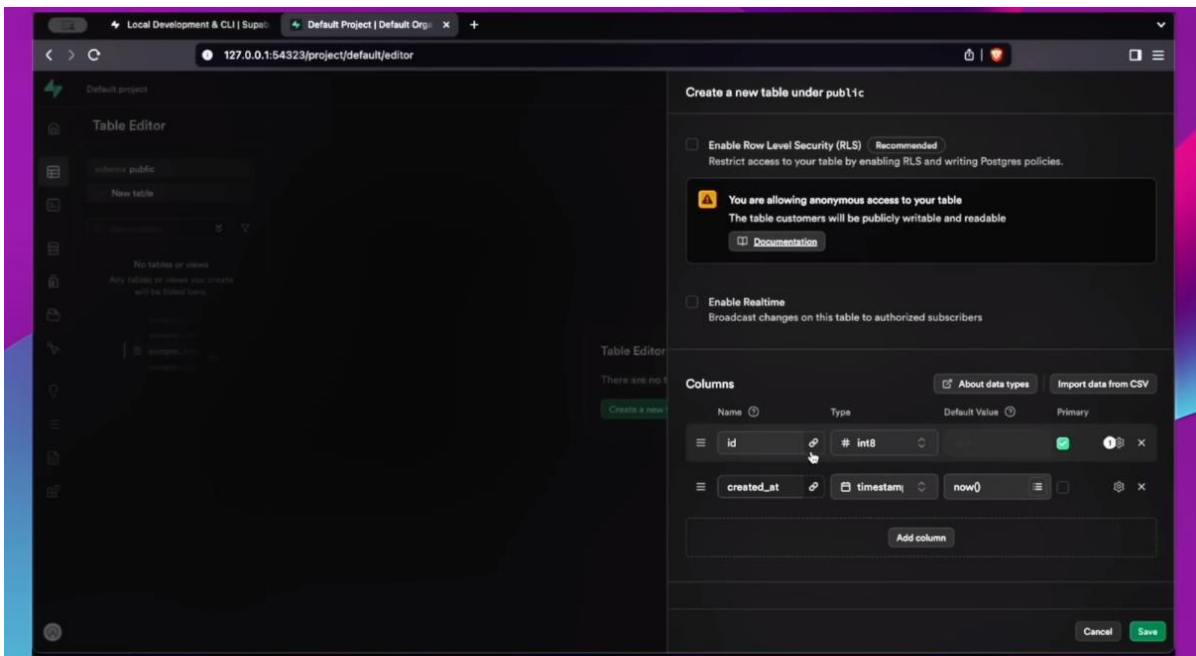
Copy the URLs into some new location



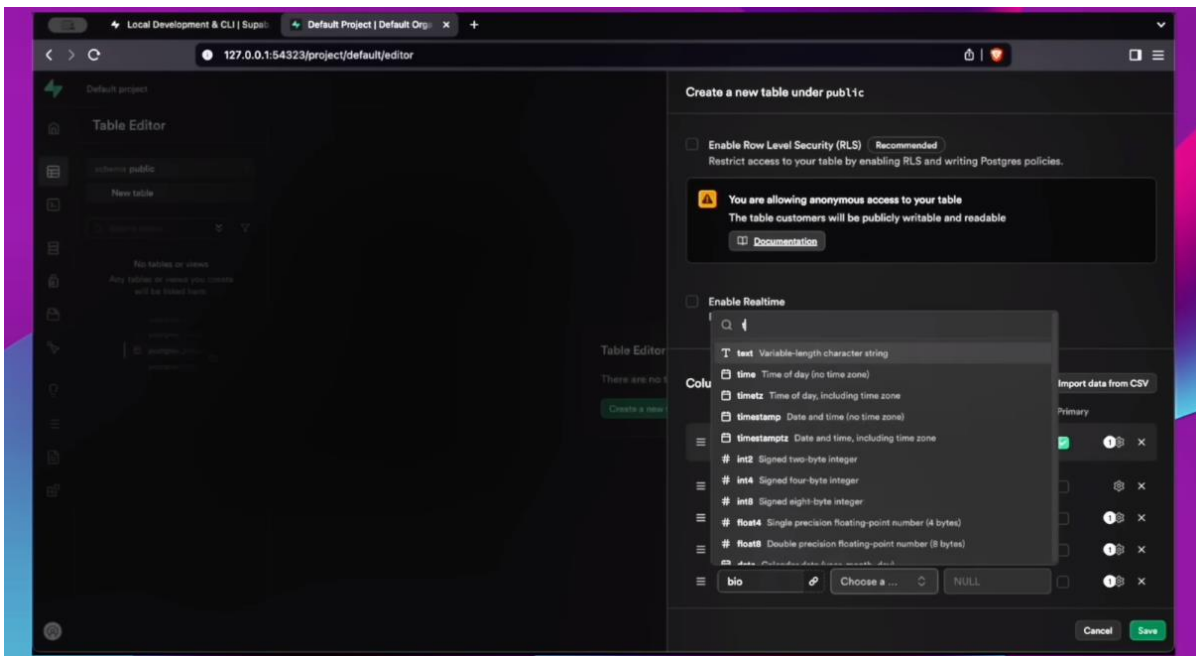
Our Supabase installation is successful and working as seen in the Studio URL above

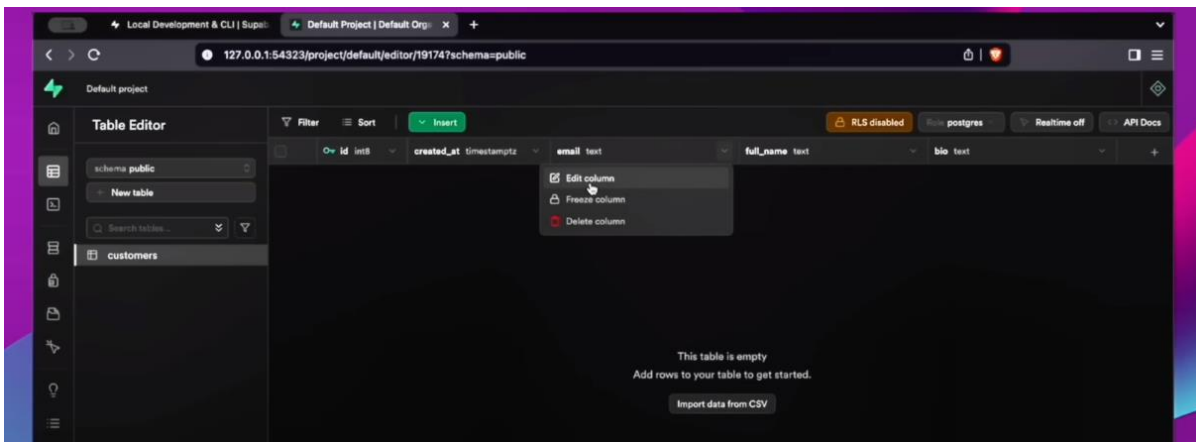
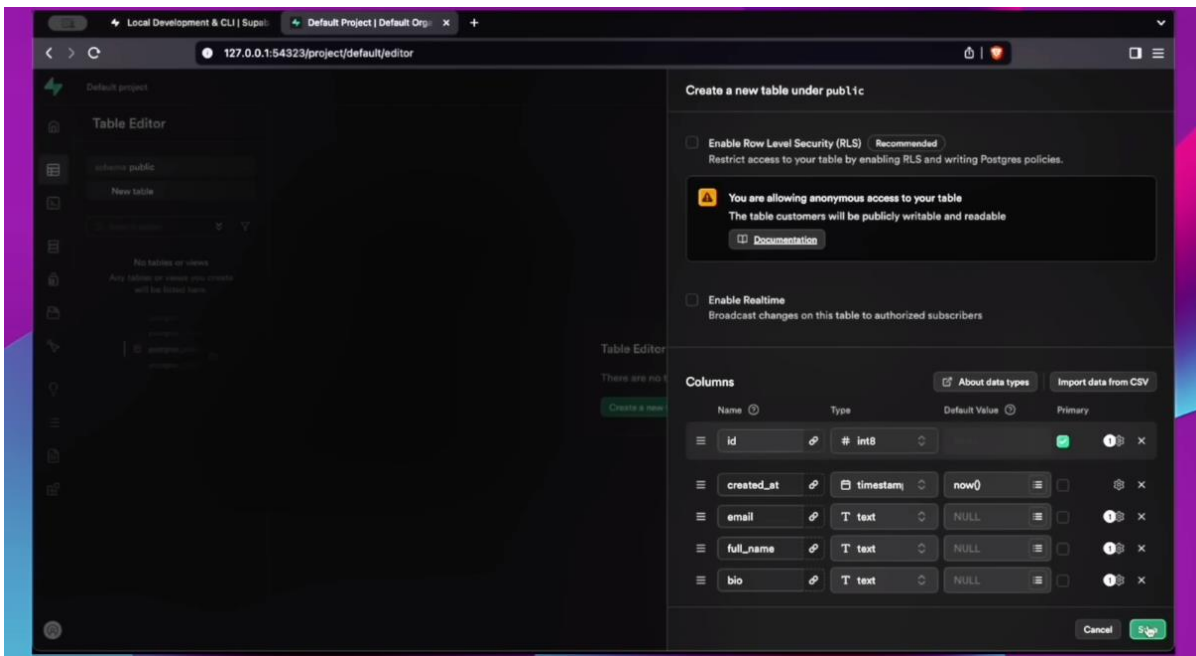




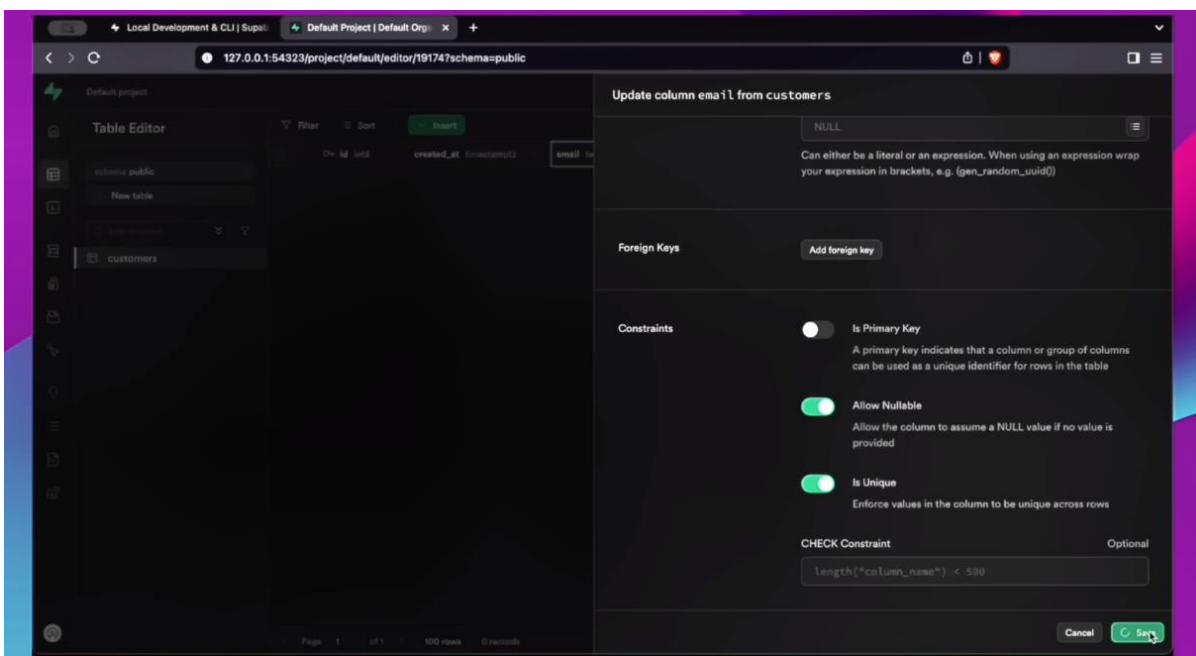


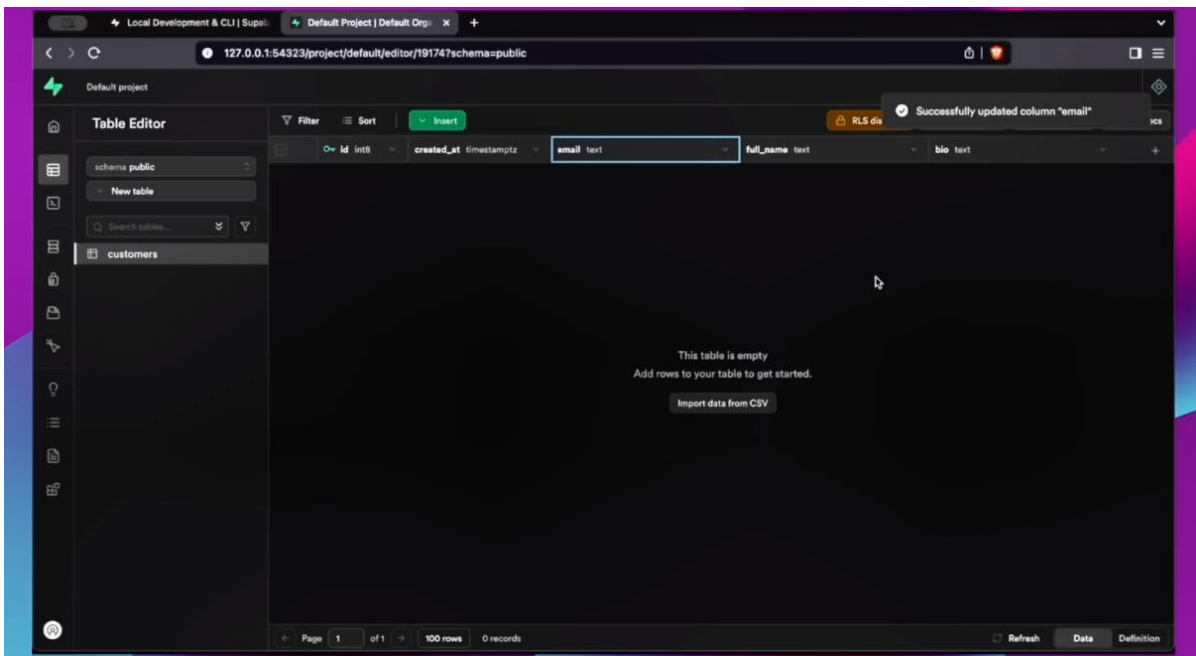
We need to add 3 new columns as below





We need to make the email column unique in our database table





We are now ready to use our Customers table in Supabase

Agent creation

```

1 import os
2 from dotenv import load_dotenv
3 from colorama import Fore
4 from dataclasses import dataclass
5 from pydantic_ai import Agent, RunContext
6 from pydantic_ai.models.groq import GroqModel
7 from supabase import create_client, Client
8
9 # Initialize model
10 load_dotenv()
11 GROQ_API_KEY: str = os.getenv("GROQ_API_KEY")
12
13 # Initialize Supabase client
14 url: str = os.getenv("SUPABASE_URL")
15 key: str = os.getenv("SUPABASE_KEY")

```

```

8
9 # Initialize model
10 load_dotenv()
11 GROQ_API_KEY: str = os.getenv("GROQ_API_KEY")
12
13 # Initialize Supabase client
14 url: str = os.getenv("SUPABASE_URL")
15 key: str = os.getenv("SUPABASE_KEY")
16 supabase: Client = create_client(url, key)
17
18 # Pydantic model for Movie
19 @dataclass
20 class Customer:
21     id: str
22     email: str
23     full_name: str
24     bio: str
25
26 if not GROQ_API_KEY:
27     raise ValueError("GROQ_API_KEY is not set")
28

```

```
17
18 # Pydantic model for Movie
19 @dataclass
20 class Customer:
21     id: str
22     email: str
23     full_name: str
24     bio: str
25
26 if not GROQ_API_KEY:
27     raise ValueError(
28         Fore.RED + "Groq API key not found. Please set the GROQ_API_KEY environment variable."
29     )
30
31 def get_model(model_name:str):
32     try:
33         model = GroqModel(
34             model_name=model_name,
35             api_key=GROQ_API_KEY)
36     except Exception as e:
37         print(Fore.RED + "Error initializing model: ", e)
38         model = None
39
40     return model
41
42 system_prompt = "You are |
```

```
41
42 system_prompt = "You are a customer service agent for a tech company. Use the tools provided to assist customers with their queries."
43 agent = Agent(model=get_model("llama-3.3-70b-versatile"), result_type=Customer, system_prompt=system_prompt)
44
45 # Seed function to create table in Supabase, populate with sample data
46 def seed_db():
47     try:
48         response = (
49             supabase.table("customers")
50             .insert([
51                 {"email": 'johndoe@gmail.com', "full_name": "John Doe", "bio": "I am a software engineer"},
52                 {"email": 'janedoe@gmail.com', "full_name": "Jane Doe", "bio": "I am a data scientist"},
53                 {"email": 'jimdove@gmail.com', "full_name": "Jim Doe", "bio": "I am a product manager"},
54             ])
55             .execute()
56         )
57         return response
58     except Exception as exception:
59         return exception
60
61 # Define the main loop
62 def ma|
```

```
60
61 # Define the main loop
62 def main_loop():
63     # Create a connection to Supabase
64     seed_db()
65     while True:
66         user_input = input(">> Enter a query (q, quit, exit to exit): ")
67         if user_input.lower() in ["quit", "exit", "q"]:
68             print("Goodbye!")
69             break
70
71         try:
72             result = agent.run_sync(user_input, deps=supabase)
73             print(Fore.YELLOW, result.data)
74         except ValueError:
75             print(Fore.RED, "No customer found with that email.")
76         except Exception as e:
77             print(Fore.RED, "An error occurred: ", e)
78
79 if __name__ == "__main__":
80     main_loop()
```



```
60
61 # Define the main loop
62 def main_loop():
63     # Create a connection to Supabase
64     seed_db()

(mynv) 12:23:13 ~/dev/tuts/full-stack-ai-masterclass/7-db-agents $ poetry run python agent.py
>> Enter a query (q, quit, exit to exit): hello
Customer(id='1234', email='customer@example.com', full_name='John Doe', bio='Hello, my name is John Doe and I am reaching out for assistance with my tech-related query.')
>> Enter a query (q, quit, exit to exit):
```

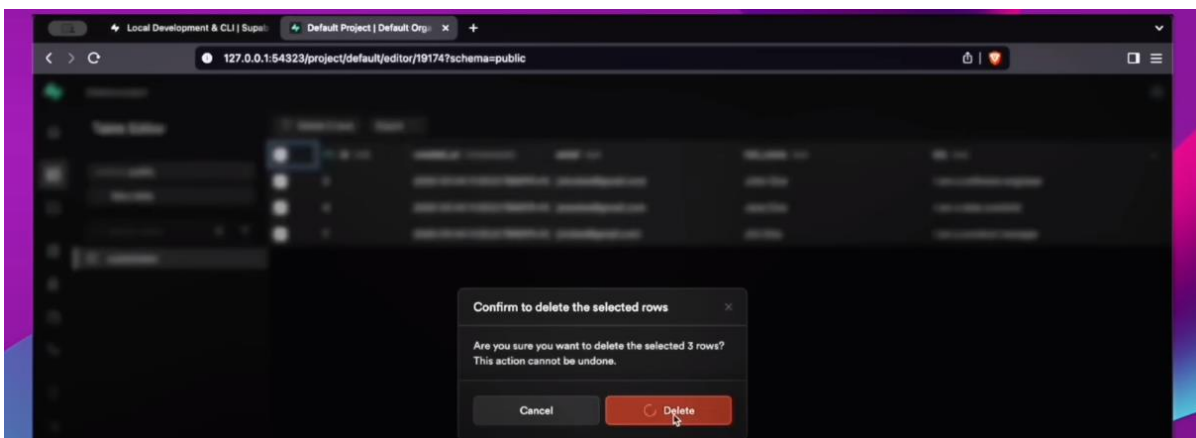
id	email	full_name	bio
5	johndoe@gmail.com	John Doe	I am a software engineer
6	janedoe@gmail.com	Jane Doe	I am a data scientist
7	jimdoe@gmail.com	Jim Doe	I am a product manager

```
60
61 # Define the main loop
62 def main_loop():
63     # Create a connection to Supabase
64     seed_db()

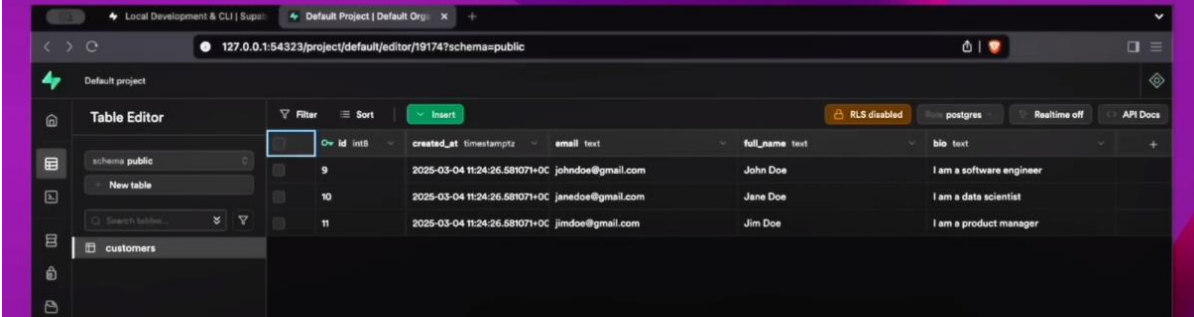
(mynv) 12:23:52 ~/dev/tuts/full-stack-ai-masterclass/7-db-agents $ poetry run python agent.py
>> Enter a query (q, quit, exit to exit): hello
Customer(id='1234', email='customer@example.com', full_name='John Doe', bio='Hello, my name is John Doe and I am reaching out for assistance with my tech-related query.')
>> Enter a query (q, quit, exit to exit):
```

id	email	full_name	bio
6	johndoe@gmail.com	John Doe	I am a software engineer
6	janedoe@gmail.com	Jane Doe	I am a data scientist
7	jimdoe@gmail.com	Jim Doe	I am a product manager

After restarting and seeding the DB again, we still have 3 rows because we made the email column unique



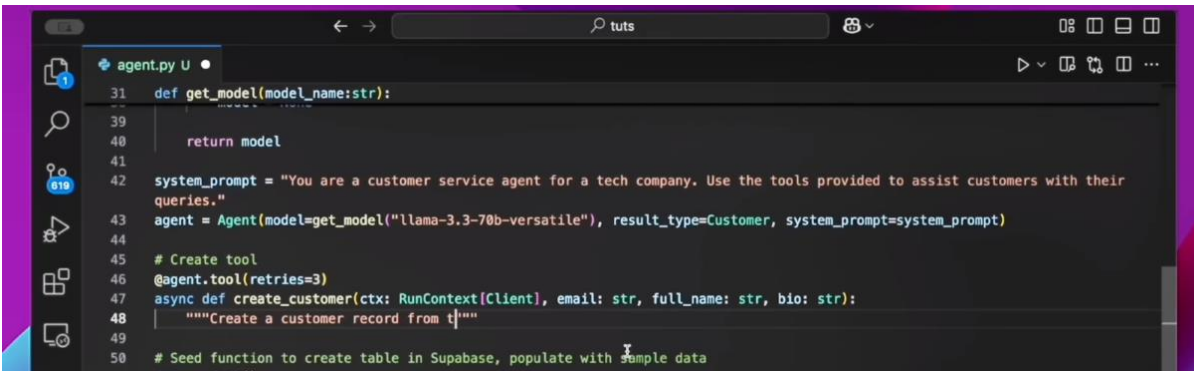
Create tool



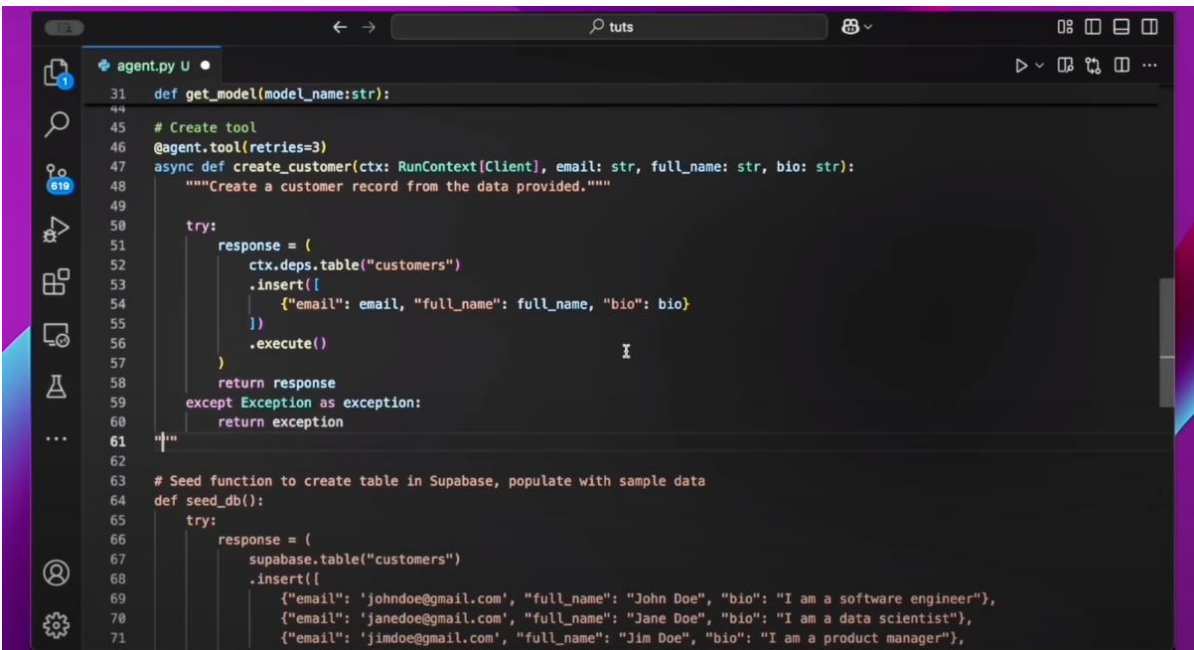
The screenshot shows the Supabase Table Editor interface. On the left, a sidebar lists the database schema, including 'public' and 'customers' tables. The main area displays the 'customers' table with columns: id, created_at, email, full_name, and bio. The table contains three rows of sample data.

id	created_at	email	full_name	bio
9	2025-03-04 11:24:26.581071+0C	john.doe@gmail.com	John Doe	I am a software engineer
10	2025-03-04 11:24:26.581071+0C	janedoe@gmail.com	Jane Doe	I am a data scientist
11	2025-03-04 11:24:26.581071+0C	jim.doe@gmail.com	Jim Doe	I am a product manager

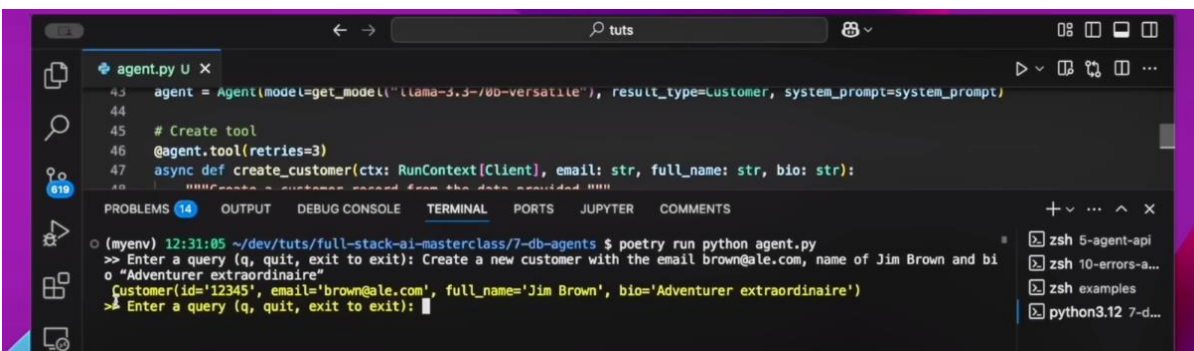
Empty the table and rerun the seeding again. We can now go ahead and start creating our agent tools as below.



```
def get_model(model_name:str):  
    return model  
  
system_prompt = "You are a customer service agent for a tech company. Use the tools provided to assist customers with their queries."  
agent = Agent(model=get_model("llama-3.3-70b-versatile"), result_type=Customer, system_prompt=system_prompt)  
  
# Create tool  
@agent.tool(retries=3)  
async def create_customer(ctx: RunContext[Client], email: str, full_name: str, bio: str):  
    """Create a customer record from t"""  
  
# Seed function to create table in Supabase, populate with sample data
```



```
    try:  
        response = (  
            ctx.deps.table("customers")  
            .insert([  
                {"email": email, "full_name": full_name, "bio": bio}  
            ])  
            .execute()  
        )  
        return response  
    except Exception as exception:  
        return exception  
  
# Seed function to create table in Supabase, populate with sample data  
def seed_db():  
    try:  
        response = (  
            supabase.table("customers")  
            .insert([  
                {"email": 'john.doe@gmail.com', "full_name": "John Doe", "bio": "I am a software engineer"},  
                {"email": 'janedoe@gmail.com', "full_name": "Jane Doe", "bio": "I am a data scientist"},  
                {"email": 'jim.doe@gmail.com', "full_name": "Jim Doe", "bio": "I am a product manager"},  
            ])  
            .execute()  
        )  
        return response  
    except Exception as exception:  
        return exception
```



```
agent = Agent(model=get_model("llama-3.3-70b-versatile"), result_type=Customer, system_prompt=system_prompt)  
  
# Create tool  
@agent.tool(retries=3)  
async def create_customer(ctx: RunContext[Client], email: str, full_name: str, bio: str):  
    """Create a customer record from the data provided"""  
  
[myenv] 12:31:05 ~/dev/tuts/full-stack-ai-masterclass/7-db-agents $ poetry run python agent.py  
>> Enter a query (q, quit, exit to exit): Create a new customer with the email brown@ale.com, name of Jim Brown and bi  
o "Adventurer extraordinaire"  
o {"customer(id='12345', email='brown@ale.com', full_name='Jim Brown', bio='Adventurer extraordinaire')"  
>> Enter a query (q, quit, exit to exit):
```

id	created_at	email	full_name	bio
9	2025-03-04 11:24:26.581071+0C	johndoe@gmail.com	John Doe	I am a software engineer
10	2025-03-04 11:24:26.581071+0C	janedoe@gmail.com	Jane Doe	I am a data scientist
11	2025-03-04 11:24:26.581071+0C	jimdoe@gmail.com	Jim Doe	I am a product manager
15	2025-03-04 11:36:32.179099+0C	brown@ale.com	Jim Brown	Adventurer extraordinaire

We have the new data inserted using the Create tool

Retrieve tool

```

47 async def create_customer(ctx: RunContext[Client], email: str, full_name: str, bio: str):
48     .execute()
49     return response
50 except Exception as exception:
51     return exception
52
53 # Retrieve tool
54 @agent.tool(retries=3)
55 async def get_customer_by_email(ctx: RunContext[Client], email: str):
56     """Retrieve a customer record from their email address."""
57
58     response = (
59         ctx.deps.table("customers")
60         .select("*")
61         .eq("email", email)
62         .execute()
63     )
64
65     if response.data:
66         return response.data[0]
67     else:
68         raise ValueError(f"No customer found with email: {email}")
69
70 # Seed function to create table in Supabase, populate with sample data
71 def seed_db():
72     try:
73         response = (
74             supabase.table("customers")
75             .insert([
76                 {"email": "johndoe@gmail.com", "full_name": "John Doe", "bio": "I am a software engineer"},
77                 {"email": "janedoe@gmail.com", "full_name": "Jane Doe", "bio": "I am a data scientist"},
78                 {"email": "jimdoe@gmail.com", "full_name": "Jim Doe", "bio": "I am a product manager"},
79             ])
80             .execute()
81         )
82         return response
83     except Exception as exception:

```

```

47 async def create_customer(ctx: RunContext[Client], email: str, full_name: str, bio: str):
48     .execute()
49     return response
50 except Exception as exception:
51     return exception
52
53 # Retrieve tool
54 @agent.tool(retries=3)
55 async def get_customer_by_email(ctx: RunContext[Client], email: str):
56     """Retrieve a customer record from their email address."""
57
58     response = (
59         ctx.deps.table("customers")
60         .select("*")
61         .eq("email", email)
62         .execute()
63     )
64
65     if response.data:
66         return response.data[0]
67     else:
68         raise ValueError(f"No customer found with email: {email}")
69
70 # Seed function to create table in Supabase, populate with sample data
71 def seed_db():
72     try:
73         response = (
74             supabase.table("customers")
75             .insert([
76                 {"email": "johndoe@gmail.com", "full_name": "John Doe", "bio": "I am a software engineer"},
77                 {"email": "janedoe@gmail.com", "full_name": "Jane Doe", "bio": "I am a data scientist"},
78                 {"email": "jimdoe@gmail.com", "full_name": "Jim Doe", "bio": "I am a product manager"},
79             ])
80             .execute()
81         )
82         return response
83     except Exception as exception:

```

```
60 | | return exception
61 |
62 | # Retrieve tool
63 | @agent.tool(retries=3)
64 | async def get_customer_by_email(ctx: RunContext[Client], email: str):
65 |     """Retrieve a customer record from their email address."""
66 |
PROBLEMS (14) OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER COMMENTS
o (myenv) 12:38:28 ~/dev/tuts/full-stack-ai-masterclass/7-db-agents $ poetry run python agent.py
>> Enter a query (q, quit, exit to exit): Retrieve customer details for customer with email brown@ale.com
Customer(id='15', email='brown@ale.com', full_name='Jim Brown', bio='Adventurer extraordinaire')
>> Enter a query (q, quit, exit to exit):
```

Filter	Sort	Insert	RLS disabled	Postgres	Realtime off	API Docs
schema public	id asc					
New table						
Search tables...						
customers	id asc					

```
60 | | return exception
61 |
62 | # Retrieve tool
63 | @agent.tool(retries=3)
64 | async def get_customer_by_email(ctx: RunContext[Client], email: str):
65 |     """Retrieve a customer record from their email address."""
66 |
PROBLEMS (14) OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER COMMENTS
o (myenv) 12:38:28 ~/dev/tuts/full-stack-ai-masterclass/7-db-agents $ poetry run python agent.py
>> Enter a query (q, quit, exit to exit): Retrieve customer details for customer with email brown@ale.com
Customer(id='15', email='brown@ale.com', full_name='Jim Brown', bio='Adventurer extraordinaire')
>> Enter a query (q, quit, exit to exit): Retrieve customer details for customer with email brown@ale.com
Customer(id='15', email='brown@ale.com', full_name='Jim Brown', bio='Great adventurer')
>> Enter a query (q, quit, exit to exit):
```

This proves that the agent is making a real-time access call to the DB and not using cached data

Update tool

```
agent.py U •
64 async def get_customer_by_email(ctx: RunContext[Client], email: str):
65     ctx.deps.table("customers")
66     .select("*")
67     .eq("email", email)
68     .execute()
69
70     if response.data:
71         return response.data[0]
72     else:
73         raise ValueError(f"No customer found with email: {email}")
74
75 # Update tool
76 @agent.tool(retries=3)
77
78 # Seed function to create table in Supabase, populate with sample data
79 def seed_db():
80     try:
81         response = (
82             supabase.table("customers")
83             .insert([
84                 {"email": 'johndoe@gmail.com', "full_name": "John Doe", "bio": "I am a software engineer"},
85                 {"email": 'janedoe@gmail.com', "full_name": "Jane Doe", "bio": "I am a data scientist"},
86                 {"email": 'jimdoe@gmail.com', "full_name": "Jim Doe", "bio": "I am a product manager"},
87             ])
88             .execute()
89         )
90     except Exception as exception:
```

```
77         raise ValueError(f"No customer found with email: {email}")
78
79 # Update tool
80 @agent.tool(retries=3)
81 async def update_customer_by_email(ctx: RunContext[Client], email: str, full_name: str, bio: str):
82     """Update a customer record from their email address."""
83
84     response = (
85         ctx.deps.table("customers")
86         .update({"full_name": full_name, "bio": bio})
87         .eq("email", email)
88         .execute()
89     )
90
91     return response
92
93 # Seed function to create table in Supabase, populate with sample data
94 def seed_db():
95     try:
96         response = (
97             supabase.table("customers")
98             .insert([
99                 {"email": 'johndoe@gmail.com', "full_name": "John Doe", "bio": "I am a software engineer"},
100                 {"email": 'janedoe@gmail.com', "full_name": "Jane Doe", "bio": "I am a data scientist"},
101                 {"email": 'jimdoe@gmail.com', "full_name": "Jim Doe", "bio": "I am a product manager"},
102             ])
103             .execute()
104         )
105     except Exception as exception:
```

Local Development & CLI | Supabase | Default Project | Default Org: x +

127.0.0.1:54323/project/default/editor/19174?schema=public

Default project

Table Editor

Filter Sort Insert RLS disabled Role postgres Realtime off API Docs

	Over	Id	int8	created_at	timestampz	email	text	full_name	text	bio	text
schema public		9		2025-03-04 11:24:26.58107+0C		johndoe@gmail.com		John Doe		I am a software engineer	
New table		10		2025-03-04 11:24:26.58107+0C		janedoe@gmail.com		Jane Doe		I am a data scientist	
Search tables...		11		2025-03-04 11:24:26.58107+0C		jimdoe@gmail.com		Jim Doe		I am a product manager	
customers		15		2025-03-04 11:36:32.179099+01		brown@ale.com		Jim Brown		Great adventurer	

```
77 | raise ValueError(f"No customer found with email: {email}")
78 |
79 | # Update tool
80 | @agent.tool(retries=3)
```

(myenv) 12:41:02 ~/dev/tuts/full-stack-ai-masterclass/7-db-agents \$ poetry run python agent.py
>> Enter a query (q, quit, exit to exit): Update customer with email brown@ale.com, to new name of James Brown and bio Godfather of soul.
Customer(id='15', email='brown@ale.com', full_name='James Brown', bio='Godfather of soul')
>> Enter a query (q, quit, exit to exit):

id	created_at	email	full_name	bio
9	2025-03-04 11:24:26.581071+0C	john.doe@gmail.com	John Doe	I am a software engineer
10	2025-03-04 11:24:26.581071+0C	janedoe@gmail.com	Jane Doe	I am a data scientist
11	2025-03-04 11:24:26.581071+0C	jim.doe@gmail.com	Jim Doe	I am a product manager

Delete tool

```
81 | async def update_customer_by_email(ctx: RunContext[Client], email: str, full_name: str, bio: str):
82 |
83 |     response = (
84 |         ctx.deps.table("customers")
85 |         .update({"full_name": full_name, "bio": bio})
86 |         .eq("email", email)
87 |         .execute()
88 |     )
89 |
90 |     return response
91 |
92 |
93 | # Delete tool
94 |
95 | # Seed function to create table in Supabase, populate with sample data
96 | def seed_db():
97 |     try:
98 |         response = (
99 |             supabase.table("customers")
100 |             .insert([
101 |                 {"email": "john.doe@gmail.com", "full_name": "John Doe", "bio": "I am a software engineer"},
102 |                 {"email": "janedoe@gmail.com", "full_name": "Jane Doe", "bio": "I am a data scientist"},
103 |                 {"email": "jim.doe@gmail.com", "full_name": "Jim Doe", "bio": "I am a product manager"},
104 |             ])
105 |             .execute()
106 |         )
107 |         return response
108 |     except Exception as exception:
109 |         return exception
110 |
```

The update works as expected

```
agent.py 3, U •
81 async def update_customer_by_email(ctx: RunContext[Client], email: str, full_name: str, bio: str):
82
83 # Delete tool
84 @agent.tool(retries=3)
85 async def delete_customer_by_email(ctx: RunContext[Client], email: str):
86     """Delete a customer record from their email address."""
87
88     response = (
89         ctx.deps.table("customers")
90         .delete()
91         .eq("email", email)
92         .execute()
93     )
94
95     return response
96
97 # Seed function to create table in Supabase, populate with sample data
98 def seed_db():
99     try:
100         response = (
101             supabase.table("customers")
102             .insert([
103                 {"email": "johndoe@gmail.com", "full_name": "John Doe", "bio": "I am a software engineer"},
104                 {"email": "janedoe@gmail.com", "full_name": "Jane Doe", "bio": "I am a data scientist"},
105                 {"email": "jimdoe@gmail.com", "full_name": "Jim Doe", "bio": "I am a product manager"},
106             ])
107             .execute()
108         )
109     except:
110         pass
111     return response
```

Table Editor

schema public

customers

id	created_at	email	full_name	bio
9	2025-03-04 11:24:26.581071+0C	johndoe@gmail.com	John Doe	I am a software engineer
10	2025-03-04 11:24:26.581071+0C	janedoe@gmail.com	Jane Doe	I am a data scientist
11	2025-03-04 11:24:26.581071+0C	jimdoe@gmail.com	Jim Doe	I am a product manager
15	2025-03-04 11:36:32.179099+01	brown@ale.com	James Brown	Godfather of soul

```
agent.py U X
91 return response
92
93 # Delete tool
```

PROBLEMS 14 OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER COMMENTS

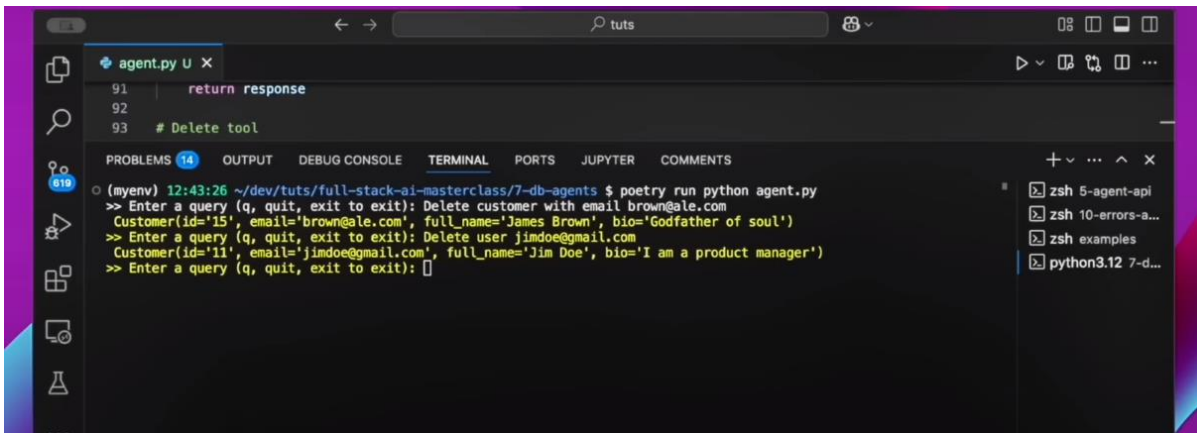
```
(myenv) 12:43:26 ~/dev/tuts/full-stack-si-masterclass/7-db-agents $ poetry run python agent.py
>> Enter a query (q, quit, exit to exit): Delete customer with email brown@ale.com
Customer(id='15', email='brown@ale.com', full_name='James Brown', bio='Godfather of soul')
>> Enter a query (q, quit, exit to exit):
```

Table Editor

schema public

customers

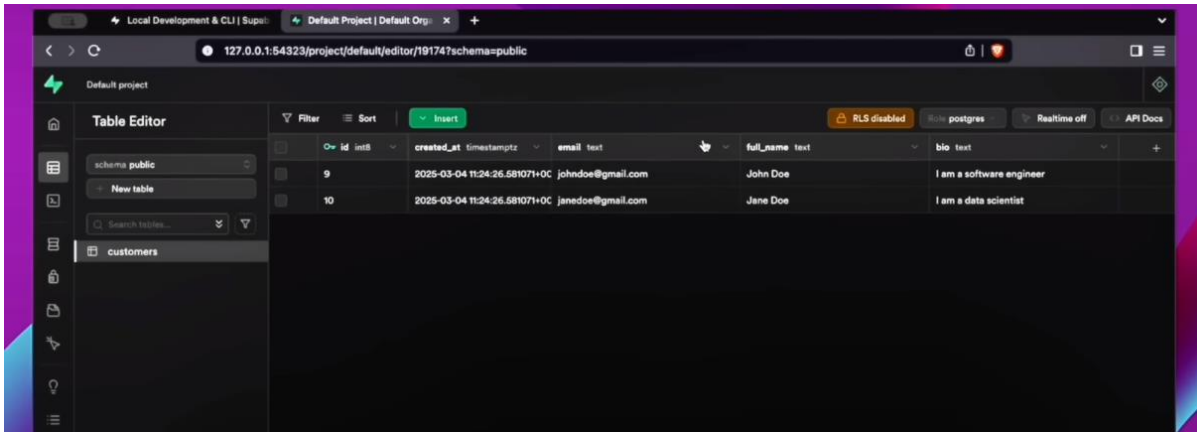
id	created_at	email	full_name	bio
9	2025-03-04 11:24:26.581071+0C	johndoe@gmail.com	John Doe	I am a software engineer
10	2025-03-04 11:24:26.581071+0C	janedoe@gmail.com	Jane Doe	I am a data scientist
11	2025-03-04 11:24:26.581071+0C	jimdoe@gmail.com	Jim Doe	I am a product manager



```
91 | return response
92
93 | # Delete tool
```

PROBLEMS (14) OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER COMMENTS

(myenv) 12:43:26 ~/dev/tuts/full-stack-ai-masterclass/7-db-agents \$ poetry run python agent.py
>> Enter a query (q, quit, exit to exit): Delete customer with email brown@ale.com
Customer(id='15', email='brown@ale.com', full_name='James Brown', bio='Godfather of soul')
>> Enter a query (q, quit, exit to exit): Delete user jimdoe@gmail.com
Customer(id='11', email='jimdoe@gmail.com', full_name='Jim Doe', bio='I am a product manager')
>> Enter a query (q, quit, exit to exit):



id	email	full_name	bio
9	2025-03-04 11:24:26.581071+0C johndoe@gmail.com	John Doe	I am a software engineer
10	2025-03-04 11:24:26.581071+0C janedoe@gmail.com	Jane Doe	I am a data scientist

This concludes our 4 DB CRUD tools for the agent

Summary

Database Agents with PydanticAI

Databases access transforms AI agents from isolated tools into integrated, enterprise-ready solutions that can scale, automate workflows, and provide real business value.

To become **truly intelligent**, AI agents must go beyond simple, stateless processing. They need to handle structured data, maintain consistency, and integrate with existing business systems. Databases play a critical role in making AI agents **scalable, reliable, and actionable**.